

My Implementation Overview

I built this AI-powered note-taking app using Laravel, Raw PHP and OpenAI. Below is a breakdown of how I implemented each feature and what I learned along the way.

1. Google OAuth Authentication

To make the login process super smooth, I added Google Sign-In. Users can log in using their Google account. I used Laravel Socialite for this.

Here's how it works:

- When a user clicks "Login with Google," they're taken to Google's sign-in screen.
- After logging in, Google sends back user info (like name and email).
- I save that info in the database (users table) and log the user into the app.
- Only logged-in users can access the dashboard or edit notes-everything is secured with Laravel middleware.

2. Note Management (CRUD)

Users can create, view, edit, and delete their notes. I made a simple and clean UI using Laravel blade so it feels super easy to handle.

Features include:

- Creating notes with a title and content.
- Viewing all notes in a dashboard (each user sees only their own).
- Deleting notes if they're no longer needed.
- On the backend, notes are stored in a notes table with a user_id reference to connect them to the right user.

3. AI Enhancement with OpenAI

This was the most interesting part. I integrated OpenAI's GPT-4.1-nano model to help users enhance their notes.

Here's how it works:

- When a user clicks the "Summarize AI" button, the app sends their note content to the Laravel backend.
- The backend calls OpenAI's API and gets back a summary.

4. Raw PHP Component

My Implementation Overview

To show I can still work with raw PHP outside Laravel, I added a standalone word count feature.

How it works:

- There's a separate file called `raw_wordcount.php` that runs pure PHP code.
- When a user clicks "Word Count," it sends the note content to this file.
- PHP calculates the word count and sends it back as a JSON response.

Even though Laravel wasn't used here, I still connected this raw PHP file to the main app so that my blade can call it.